¡Programación Orientada a Objetos!

Creando métodos y pruebas para nuestro Auto a Control Remoto 🚗💨

¿Qué son los Métodos de Acceso?

También conocidos como **"Getters"**, son métodos especiales que nos permiten conocer el valor de un atributo privado de un objeto desde afuera. Generalmente inician con la palabra `get` obtener en inglés).

```
public String getNombre() {
   return this.nombre;
}
```



El Reto del Día 🏎️

Vamos a diseñar e implementar una clase en Java que modele un **Auto de Control Remoto**. Imagina que has comprado un auto nuevo. Éste cuenta con una pantalla LED inteligente que muestra dos cosas muy importantes:

- La **distancia total** que ha recorrido.
- El porcentaje de **batería restante**.



Reglas de Funcionamiento



1. Energía Inicial

Al salir de su caja, el auto inicia con el **100% de batería**. ¡Listo para la acción!



2. Al Conducir

Cada vez que lo conduces avanza exactamente **20 metros**, pero consume **1% de su batería**.



3. Sin Energía

Si la batería llega a **0%**, el auto ya no puede avanzar, por más que intentemos conducirlo.

Diseñando Nuestra Clase (UML)

Visibilidad / Tipo	Nombre del Elemento	Descripción
`private int`	**distanciaRecorrida**	Atributo: Guarda los metros que ha avanzado.
`private int`	**bateriaRestante**	Atributo: Guarda el porcentaje de carga actual.
`public`	**AutoRemoto()**	Constructor: Prepara el auto nuevo.
`public void`	**conducir()**	Método: Ejecuta la acción de avanzar.
`public int`	**getBateriaRestante()**	Getter: Nos dice cuánta batería le queda.
`public int`	**getDistanciaRecorrida()**	Getter: Nos dice cuántos metros ha avanzado.

Programando: Atributos y Getters

1. Atributos y Constructor

```
public class AutoRemoto {
  private int distanciaRecorrida;
  private int bateriaRestante;

  public AutoRemoto() {
    this.distanciaRecorrida = 0;
    this.bateriaRestante = 100;
  }
}
```

Al crear un auto nuevo, la distancia es cero y la batería está llena.

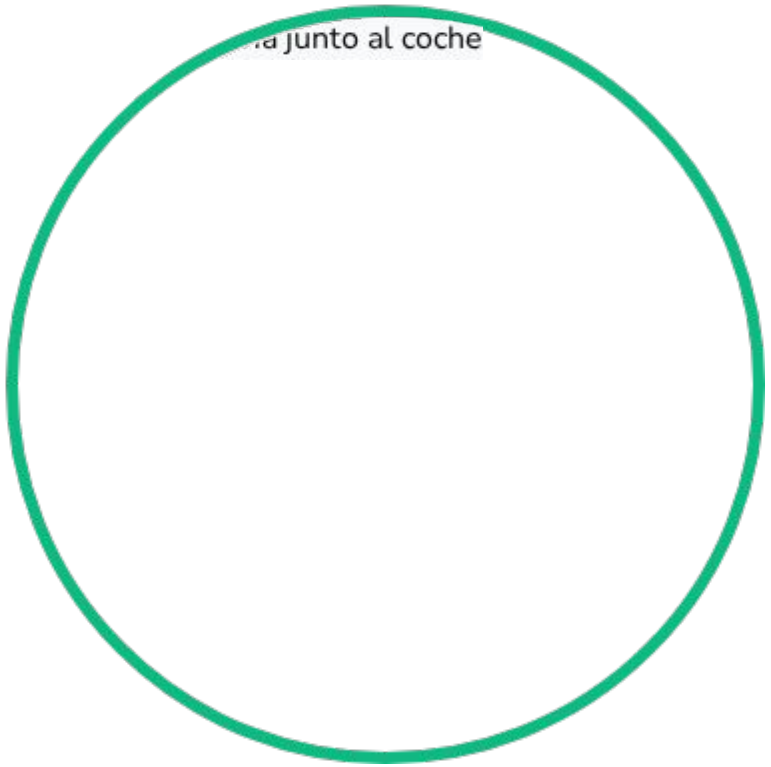
2. Nuestros Getters

```
public int getBateriaRestante() {
  return this.bateriaRestante;
}

public int getDistanciaRecorrida() {
  return this.distanciaRecorrida;
}
```

Estos métodos nos permiten "ver" la pantalla LED de nuestro auto.

La Acción: Método `conducir()`



Dando marcha al auto

Antes de avanzar, debemos preguntar: **¿Tenemos batería?**

```
public void conducir() {
  if (this.bateriaRestante > 0) {
    this.distanciaRecorrida += 20;
    this.bateriaRestante -= 1;
  }
}
```

Si la batería es mayor a 0, sumamos 20 metros a la distancia y restamos 1% a la batería.



¡Hora de las Pruebas!

Vamos a escribir código de pruebas en JUnit para asegurarnos de que nuestro auto funcione perfectamente y no nos deje tirados.

Pruebas de Auto Nuevo (Fábrica)

- **Prueba 1: Distancia Inicial** Verificamos que al crear el auto, la distancia recorrida sea exactamente cero.

- **Prueba 2: Batería Inicial** Verificamos que la batería disponible en un auto nuevo esté al 100%.

Pruebas de Auto en Acción

- **Prueba 3: Distancia al Conducir** Llamamos a conducir() una vez, y verificamos que el auto haya avanzado 20 metros.

- **Prueba 4: Batería al Conducir** Después de usar conducir() una vez, comprobamos que la energía haya bajado a 99%.

Prueba 5: El Límite de Energía

100

Veces Conducidas

¿Qué pasa si la batería se agota?

Nuestra última prueba usa un ciclo while para llamar al método conducir() exactamente **100 veces** y verificamos que la batería llege a 0 y que nuestro funciona protegiendo números negativos

.

Prueba 6: El Límite de Distacia

200

Veces Conducidas

¿Qué pasa si la batería se agota?

Nuestra última prueba usa un ciclo while para llamar al método conducir() exactamente **200 veces** y verificamos que la distancia no supere 2000.

.

¿Preguntas? 

¡Hemos completado con éxito nuestro auto a control remoto!



Image Sources



https://png.pngtree.com/png-clipart/20250712/original/pngtree-blue-3d-magnifying-glass-icon-with-modern-design-and-transparent-background-png-image_21234149.png

Source: pngtree.com



https://static.vecteezy.com/system/resources/previews/069/733/316/non_2x/red-toy-car-miniature-vehicle-plastic-white-racing-stripe-isolated-playful-children-small-modern-fun-cheerful-design-for-kids-png.png

Source: www.vecteezy.com



https://lookaside.fbsbx.com/lookaside/crawler/media/?media_id=1536546985184116

Source: www.facebook.com